Post Lab Tasks: Pointer's Practice

COURSE CODE: C1002

SUBJECT: PROGRAMMING FUNDAMENTALS

INSTRUCTOR: MR. SHAYAN ALI

NAME: AMNA KHAN

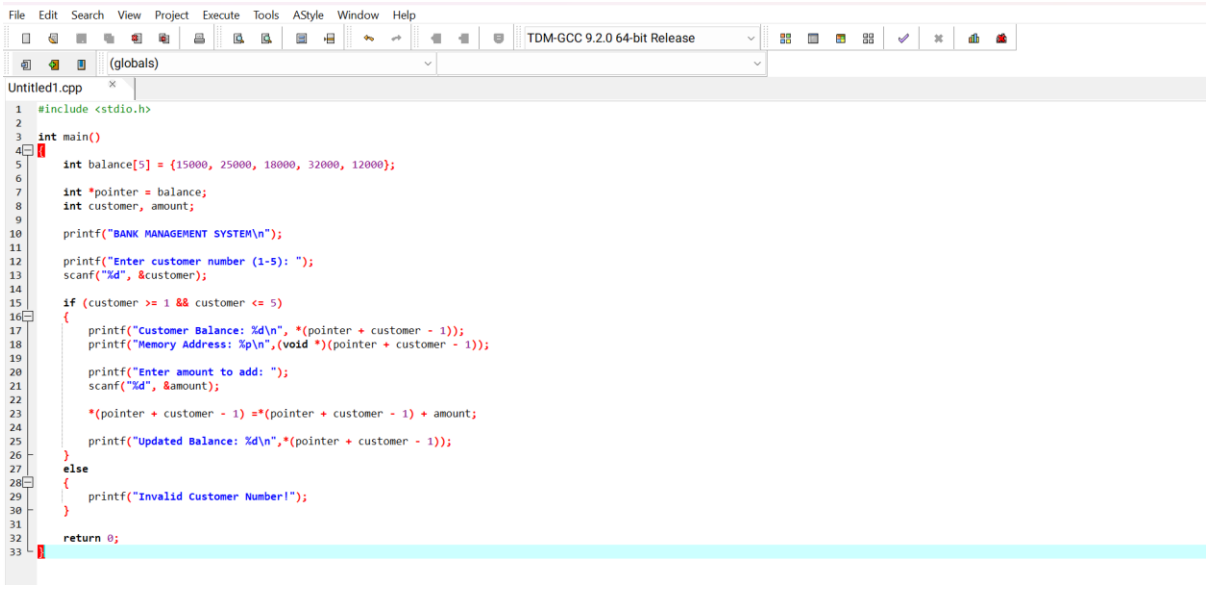
ROLL NO/ID: 26K-3091

CLASS/SECTION: C

DATE OF SUBMISSION: 9-NOV'2026

Task 1 (Basic Pointers and Account Balance Management):

Create an array named balance containing {15000, 25000, 18000, 32000, 12000}. Use a pointer to access a selected customer's balance, display its value and memory address, and update the balance through pointer dereferencing.




```c
#include <stdio.h>

int main()
{
    int balance[5] = {15000, 25000, 18000, 32000, 12000};

    int *pointer = balance;
    int customer, amount;

    printf("BANK MANAGEMENT SYSTEM\n");

    printf("Enter customer number (1-5): ");
    scanf("%d", &customer);

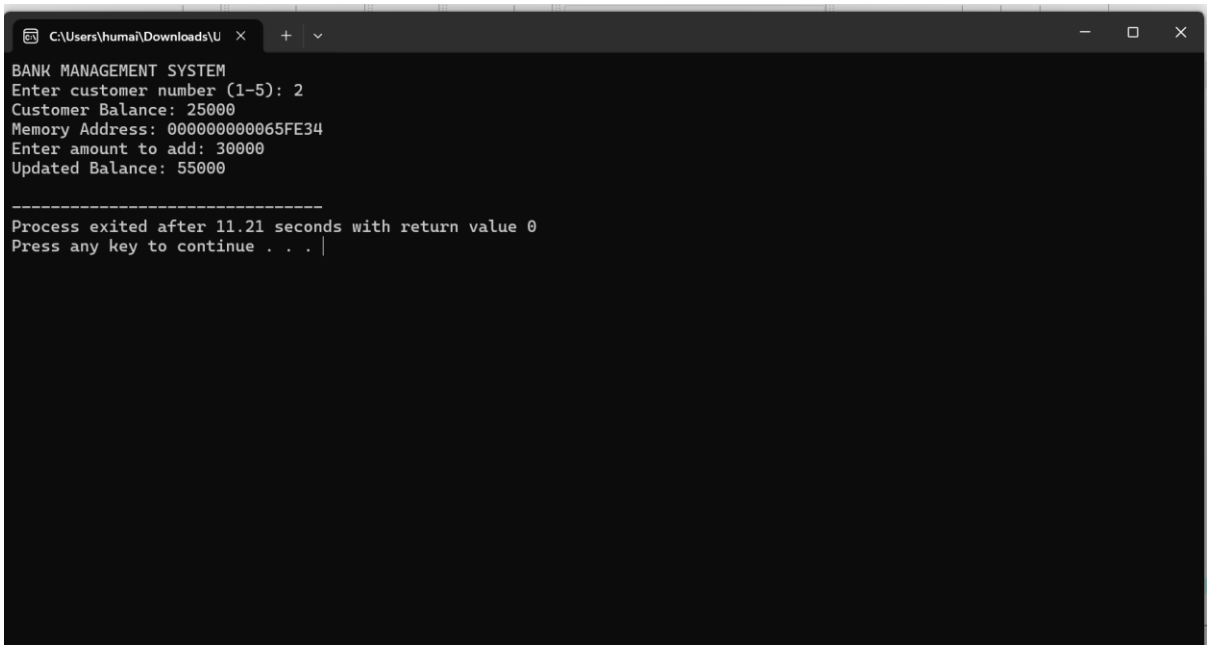
    if (customer >= 1 && customer <= 5)
    {
        printf("Customer Balance: %d\n", *(pointer + customer - 1));
        printf("Memory Address: %p\n",(void *)(pointer + customer - 1));

        printf("Enter amount to add: ");
        scanf("%d", &amount);

        *(pointer + customer - 1) =*(pointer + customer - 1) + amount;

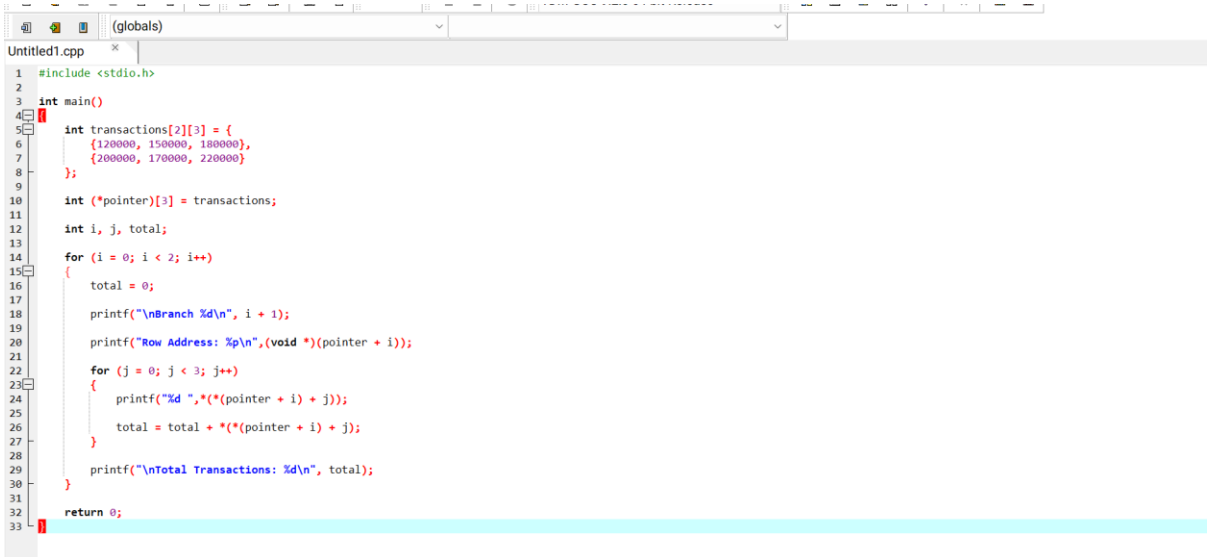
        printf("Updated Balance: %d\n",*(pointer + customer - 1));
    }
    else
    {
        printf("Invalid Customer Number!");
    }

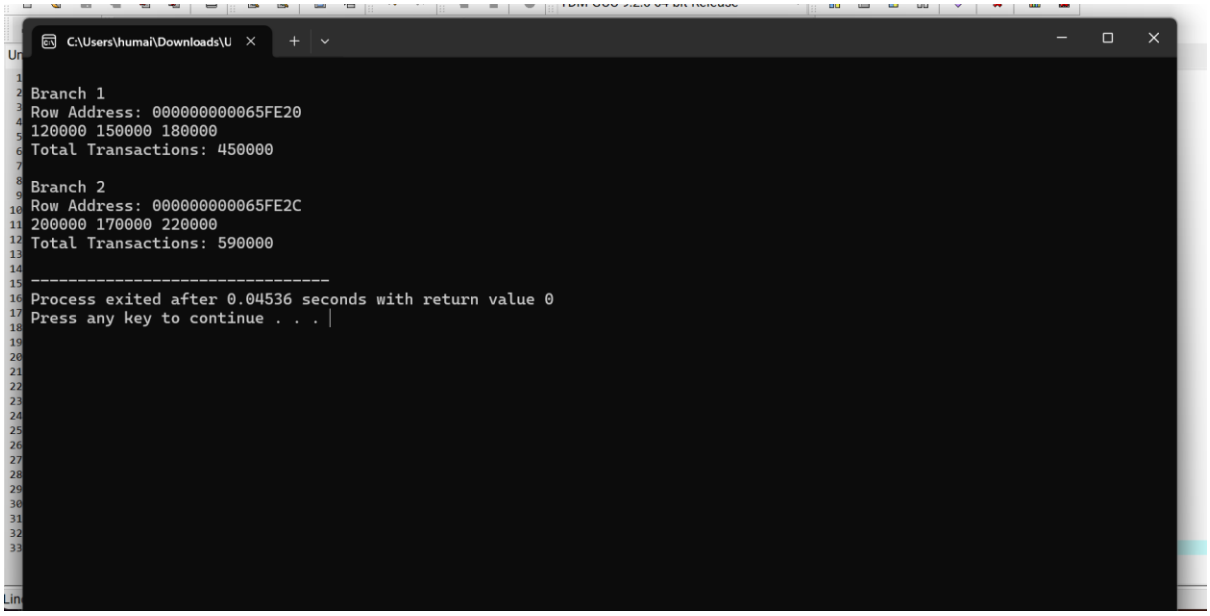
    return 0;
}
```



Task 2 (Pointer to a 2D Array):

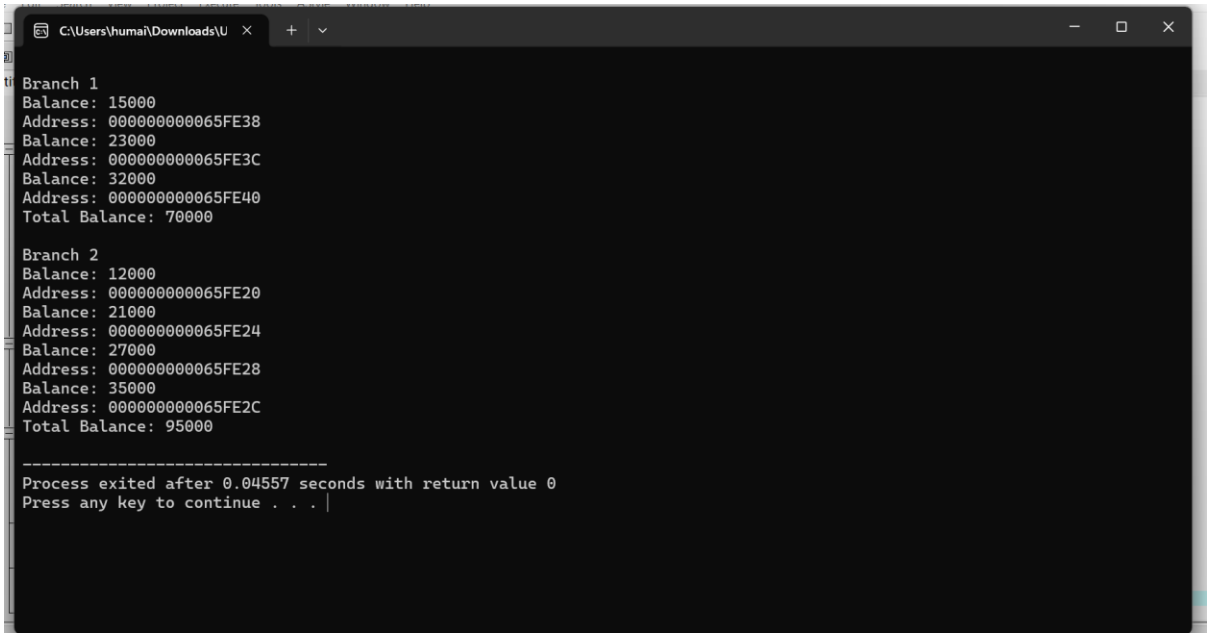
Create the transactions array containing {{120000, 150000, 180000}, {200000, 170000, 220000}}. Use a pointer to an array of three integers and pointer arithmetic to access both branches' records. Display the transaction values, row addresses, and total transactions of each branch.



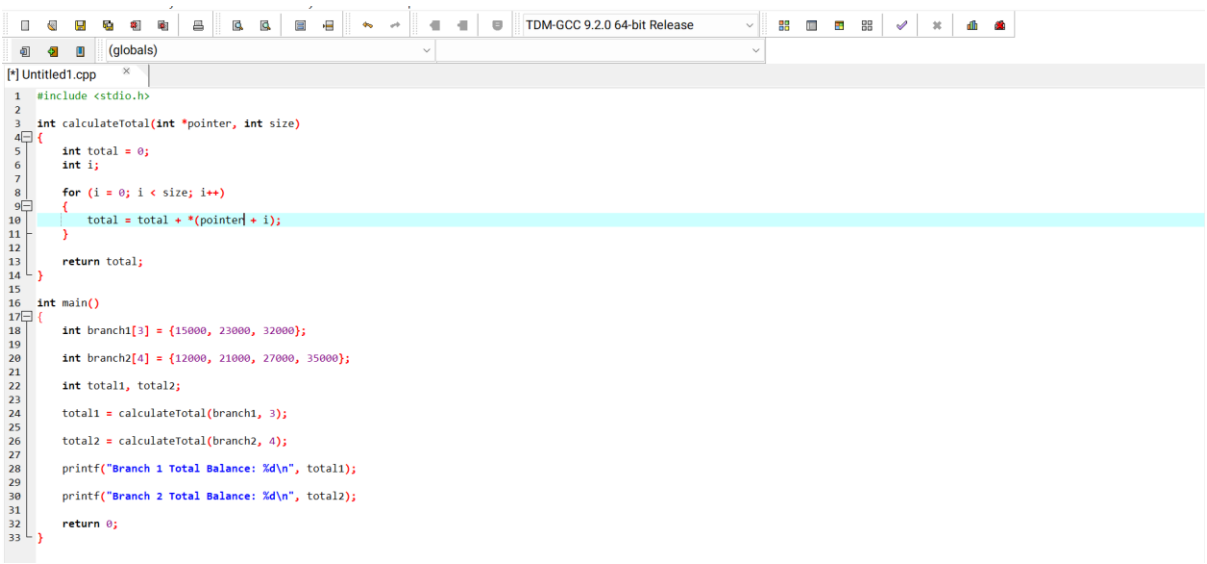


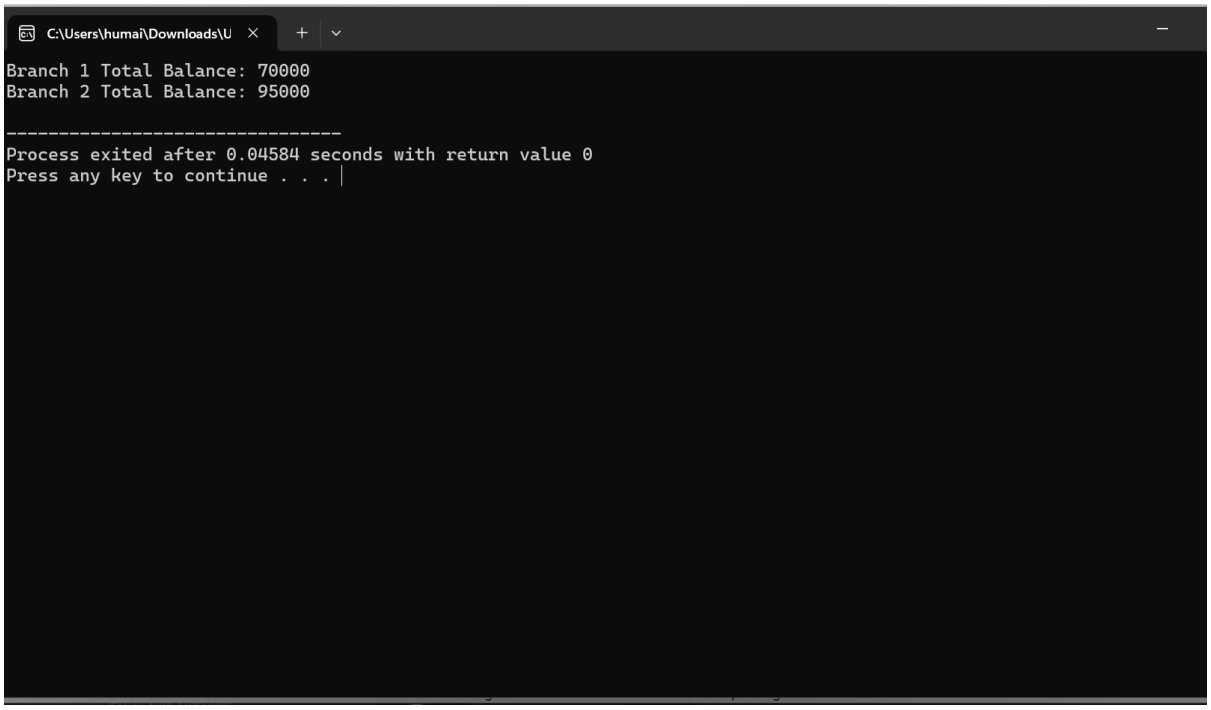
Task 3 (Array of Pointers): Create two customer account arrays: Branch 1 {15000, 23000, 32000} and Branch 2 {12000, 21000, 27000, 35000}. Use an array of pointers to access their balances, display values and memory addresses, and calculate the total balance of each branch.



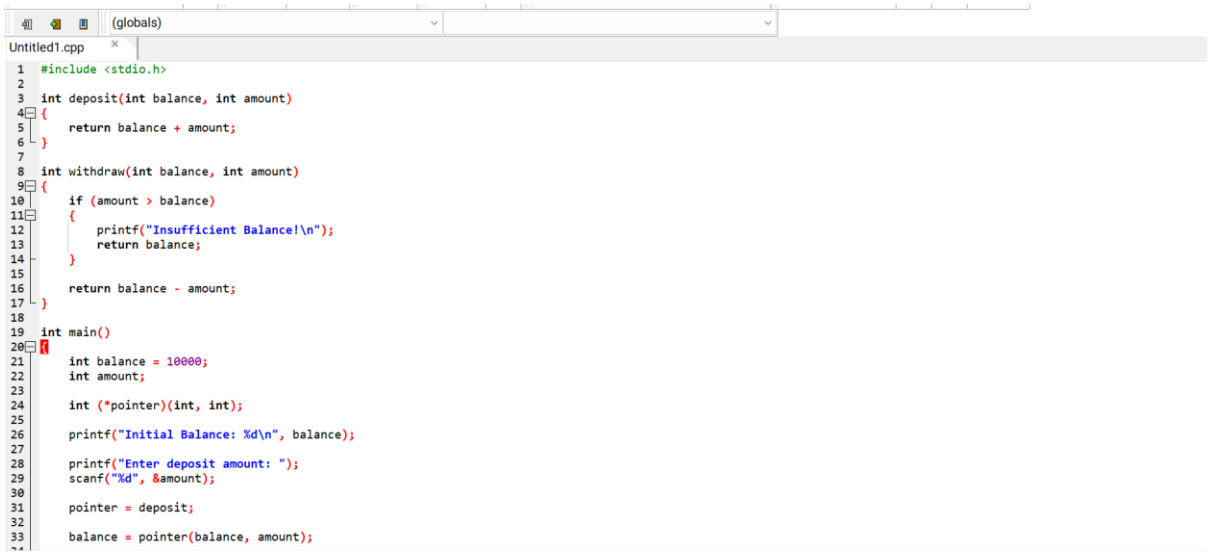


Task 4 (Passing an Array to a Function): Create calculateTotal() that receives an integer pointer and array size. Use pointer notation to calculate and return the total balance. Call this function for both branch account arrays.





Task 5 (Function Pointers): Create deposit() and withdraw() functions, each accepting two integers and returning an integer result. Use a function pointer to call both operations and display the updated balance.



Untitled1.cpp

```
    return balance - amount;
}

int main()
{
    int balance = 10000;
    int amount;

    int (*pointer)(int, int);

    printf("Initial Balance: %d\n", balance);

    printf("Enter deposit amount: ");
    scanf("%d", &amount);

    pointer = deposit;

    balance = pointer(balance, amount);

    printf("After Deposit: %d\n", balance);

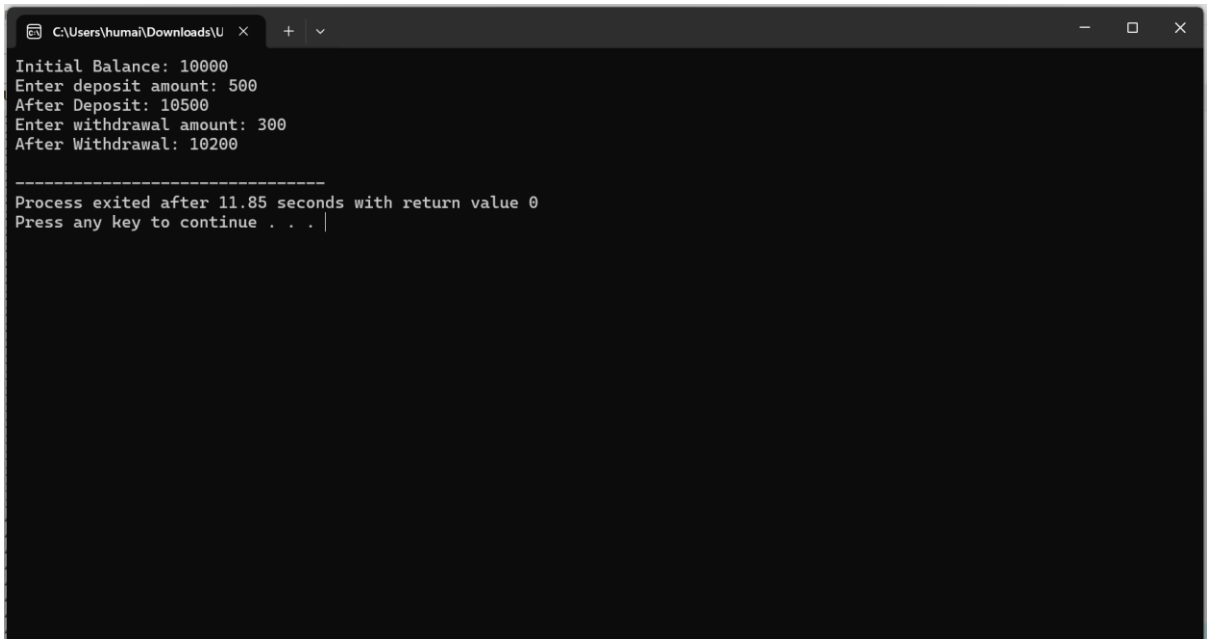
    printf("Enter withdrawal amount: ");
    scanf("%d", &amount);

    pointer = withdraw;

    balance = pointer(balance, amount);

    printf("After Withdrawal: %d\n", balance);

    return 0;
}
```



Task 6 (Passing Function Pointers as Arguments): Create processTransaction() that receives two integers and a function pointer. Use the received pointer to call either deposit() or withdraw() and return the result. Call this function from main() for both operations.

(globals)

Untitled1.cpp

```c
#include <stdio.h>

int deposit(int balance, int amount)
{
    return balance + amount;
}

int withdraw(int balance, int amount)
{
    if (amount > balance)
    {
        printf("Insufficient Balance!\n");
        return balance;
    }

    return balance - amount;
}

int processTransaction(int balance, int amount,int (*pointer)(int, int))
{
    return pointer(balance, amount);
}

int main()
{
    int balance = 10000;
    int amount;

    printf("Initial Balance: %d\n", balance);

    printf("Enter deposit amount: ");
    scanf("%d", &amount);
```

(globals)

Untitled1.cpp

```c
    }

    return balance - amount;
}

int processTransaction(int balance, int amount,int (*pointer)(int, int))
{
    return pointer(balance, amount);
}

int main()
{
    int balance = 10000;
    int amount;

    printf("Initial Balance: %d\n", balance);

    printf("Enter deposit amount: ");
    scanf("%d", &amount);

    balance = processTransaction(balance, amount, deposit);

    printf("After Deposit: %d\n", balance);

    printf("Enter withdrawal amount: ");
    scanf("%d", &amount);

    balance = processTransaction(balance, amount, withdraw);

    printf("After Withdrawal: %d\n", balance);

    return 0;
}
```

C:\Users\humai\Downloads\U

```
Initial Balance: 10000
Enter deposit amount: 4000
After Deposit: 14000
Enter withdrawal amount: 600
After Withdrawal: 13400

--------------------------------
Process exited after 7.232 seconds with return value 0
Press any key to continue . . .
```